Assignment 5 (Cloud Services & Security)

PRN No. - 260840127015 Name - Khendake Pradip Goraksh Course - PGCP-HPCSA

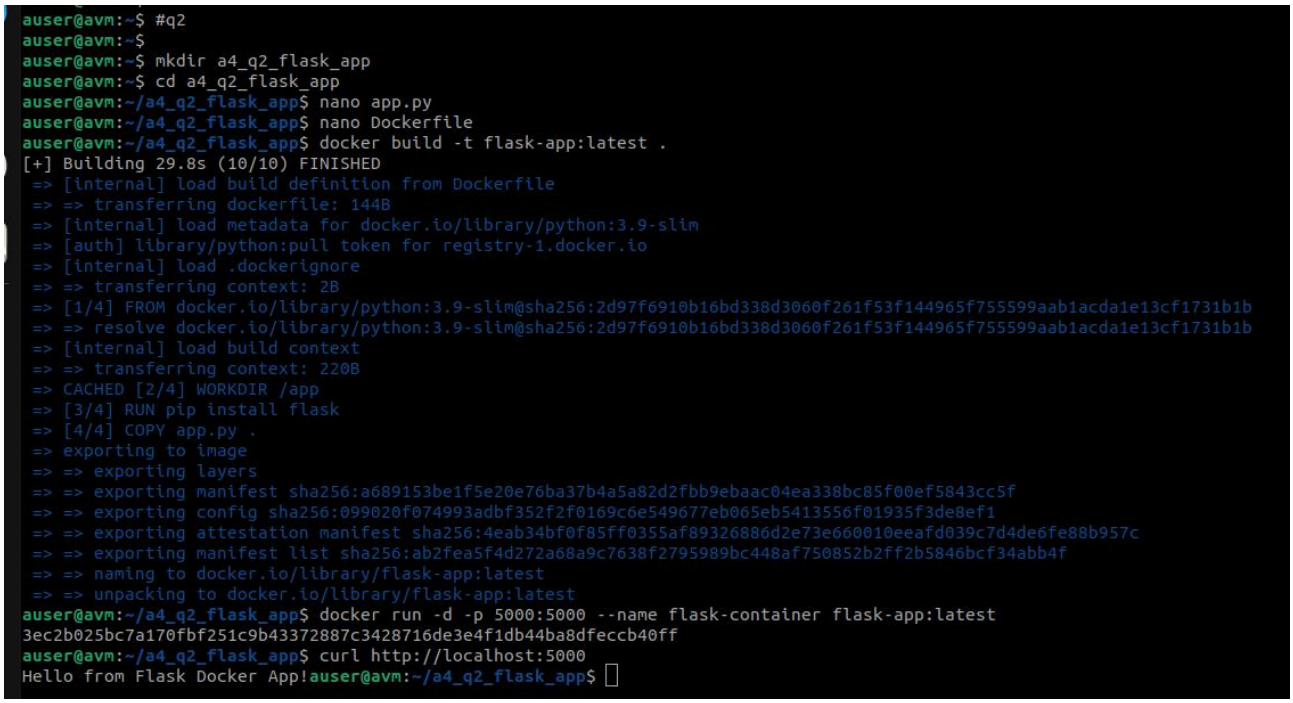
1\. Create a GitHub repository containing a simple Python application and a `Dockerfile`. Clone the repository to your local machine.

```
auser@avm:~$ mkdir a4_q1_flask-docker-demo
auser@avm:~$ cd a4_q1_flask-docker-demo
auser@avm:~/a4_q1_flask-docker-demo$ nano app.py
auser@avm:~/a4_q1_flask-docker-demo$ nano Dockerfile
auser@avm:~/a4_q1_flask-docker-demo$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/auser/a4_q1_flask-docker-demo/.git/
auser@avm:~/a4_q1_flask-docker-demo$ git add .
auser@avm:~/a4_q1_flask-docker-demo$ git commit "Initial commit with app and Dockerfile
> "
error: pathspec 'Initial commit with app and Dockerfile
' did not match any file(s) known to git
auser@avm:~/a4_q1_flask-docker-demo$ git commit "Initial commit with app and Dockerfile"
error: pathspec 'Initial commit with app and Dockerfile' did not match any file(s) known to git
auser@avm:~/a4_q1_flask-docker-demo$ git commit -m "Initial commit with app and Dockerfile"
[master (root-commit) 6033ea4] Initial commit with app and Dockerfile
 2 files changed, 6 insertions(+)
 create mode 100644 Dockerfile
 create mode 100644 app.py
auser@avm:~/a4_q1_flask-docker-demo$ git branch -M main
auser@avm:~/a4_q1_flask-docker-demo$ git remote add origin https://github.com/himanic72538/a4_q1_flask-docker-demo.git
auser@avm:~/a4_q1_flask-docker-demo$ git push -u origin main
Username for 'https://github.com': himanic72538
Password for 'https://himanic72538@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 382 bytes | 382.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/himanic72538/a4_q1_flask-docker-demo.git
 * [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
auser@avm:~/a4_q1_flask-docker-demo$ cd
auser@avm:~$ ^[[200~cd ~
cd: command not found
auser@avm:~$ git clone https://github.com/himanic72538/a4_q1_flask-docker-demo.git cloned_repo
Cloning into 'cloned_repo'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 0), reused 4 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.
auser@avm:~$
```

```
auser@avm: ~/a4_q1_flask-docker-demo
 GNU nano 6.2                                                app.py
print("Hello from Assignment 4 GitHub Repo!")
```

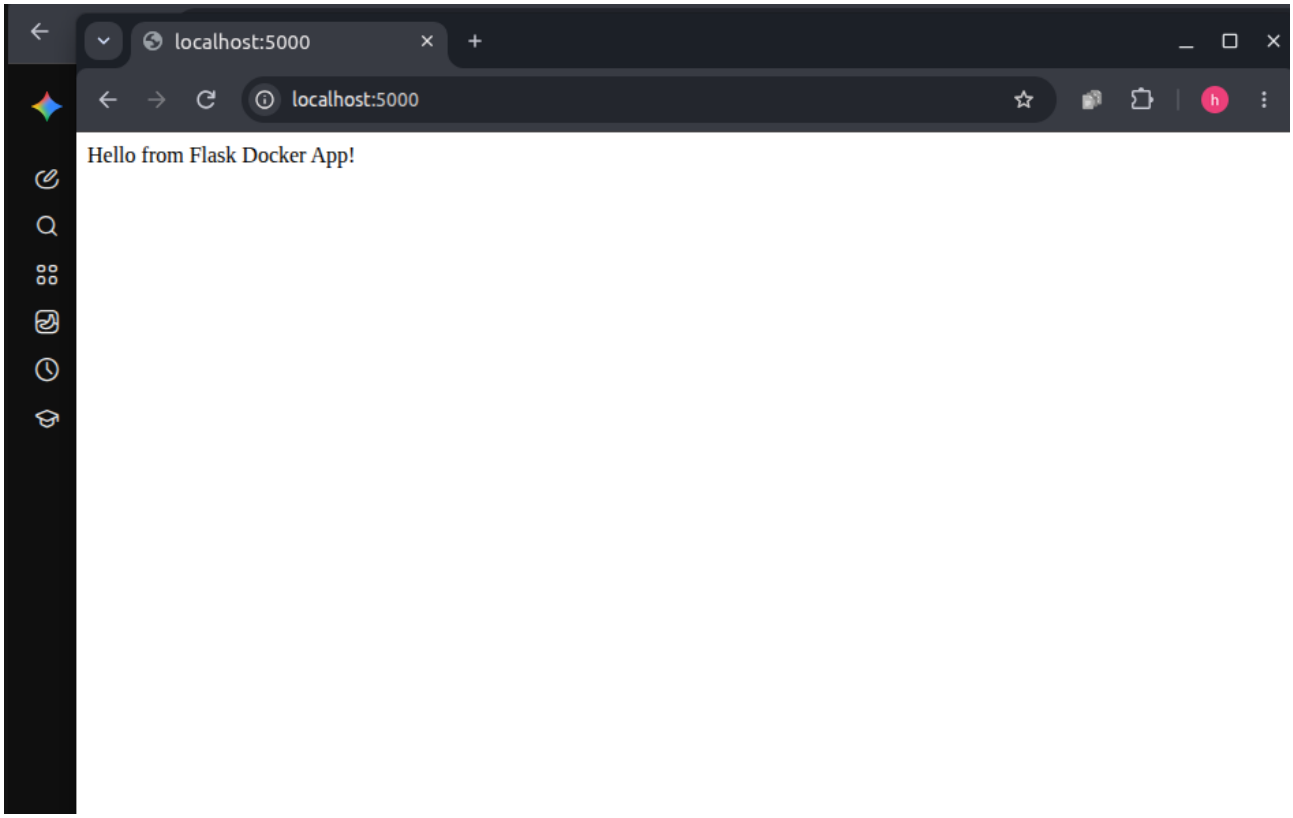


2\. Write a `Dockerfile` for a simple Python Flask application. Build a Docker image and run a container.

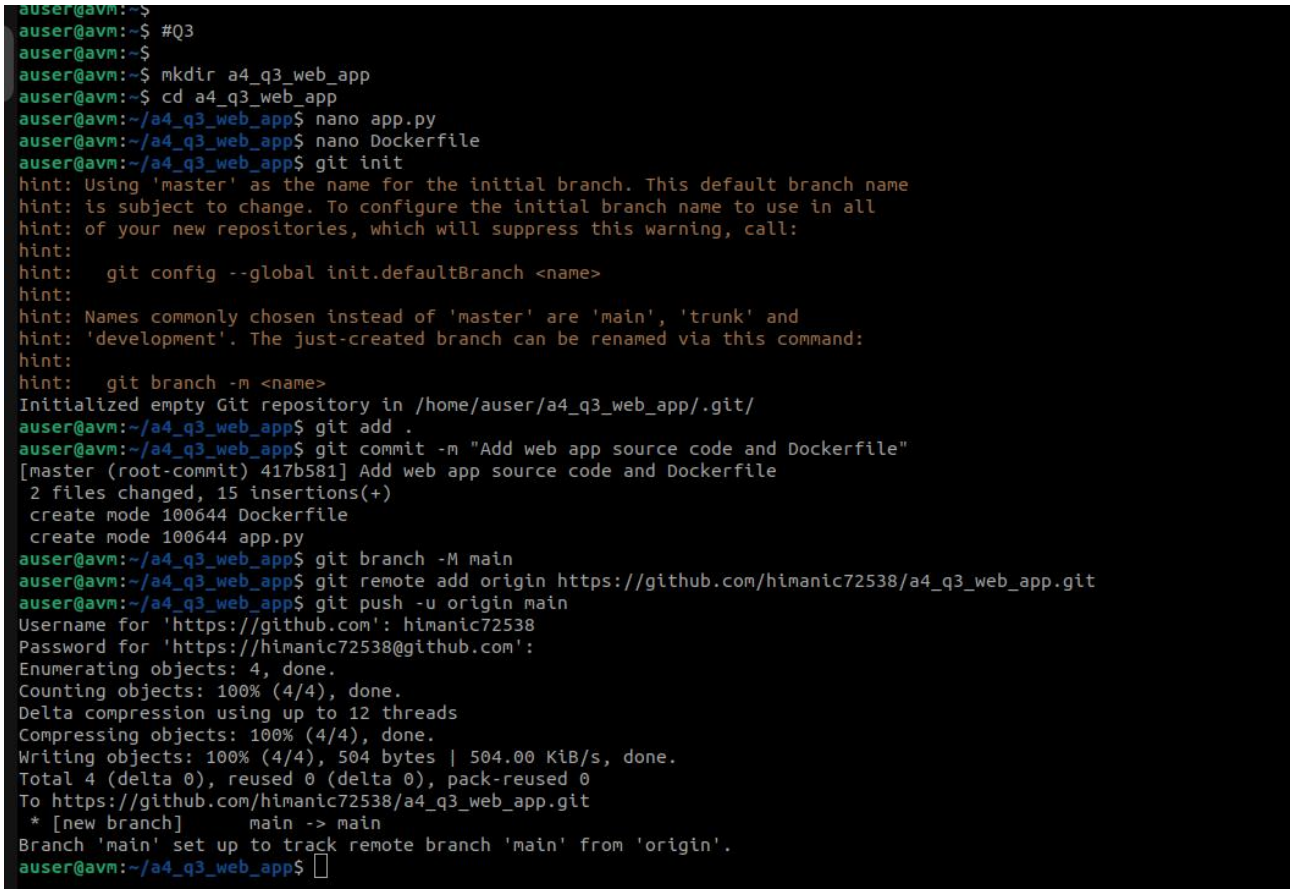


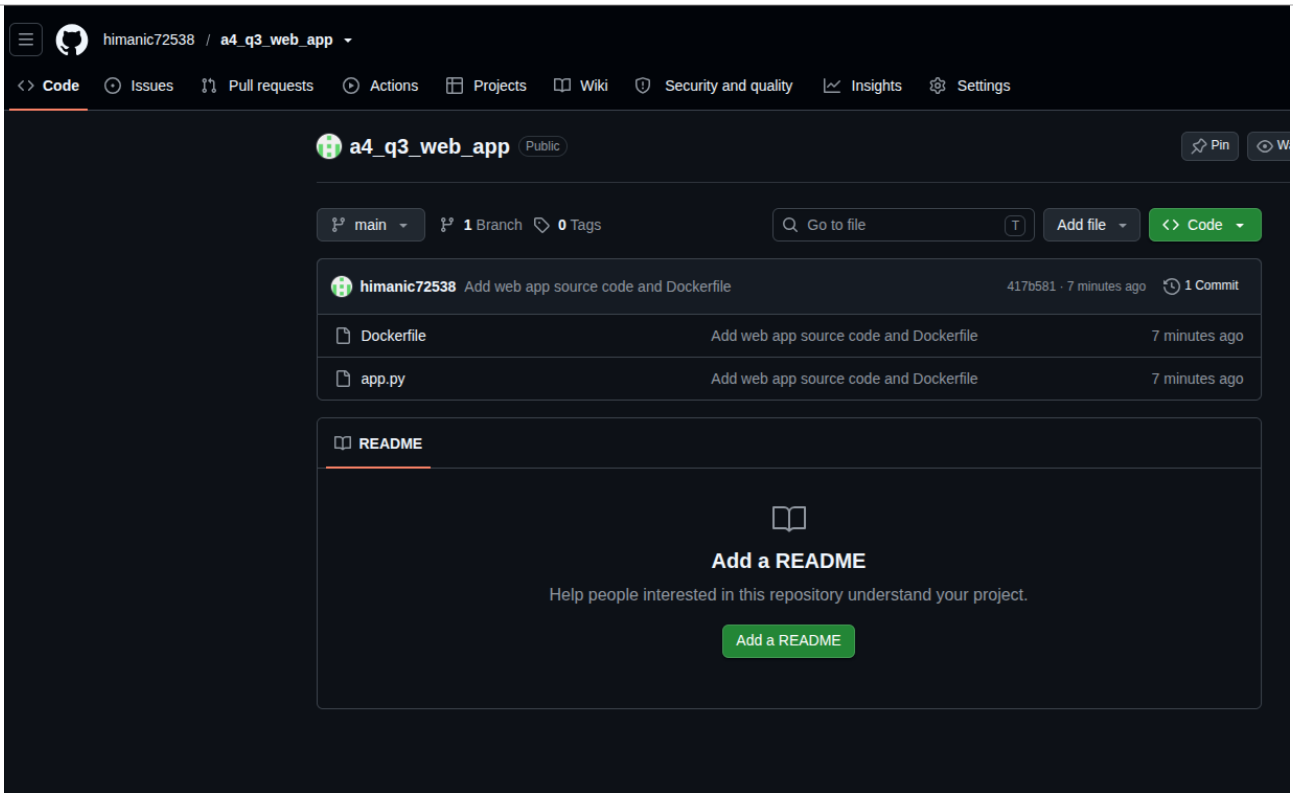


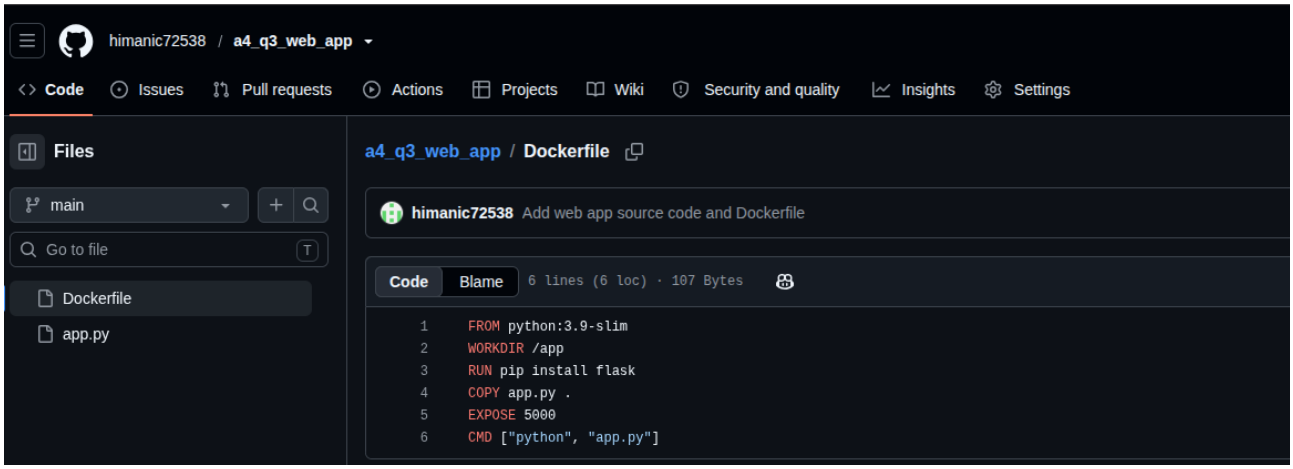

auser@avm: ~/a4_q2_flask_app
GNU nano 6.2 Dockerfile
FROM python:3.9-slim
WORKDIR /app
RUN pip install flask
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]

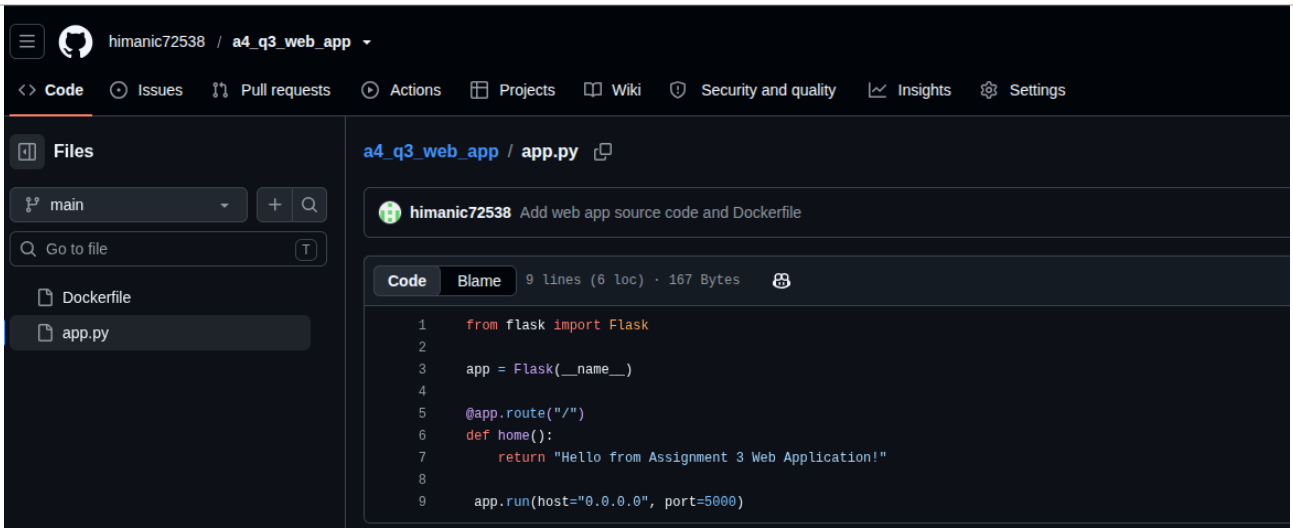

localhost:5000
localhost:5000
Hello from Flask Docker App!

3\. Create a simple web application, upload its source code and `Dockerfile` to GitHub, and verify the repository contents.

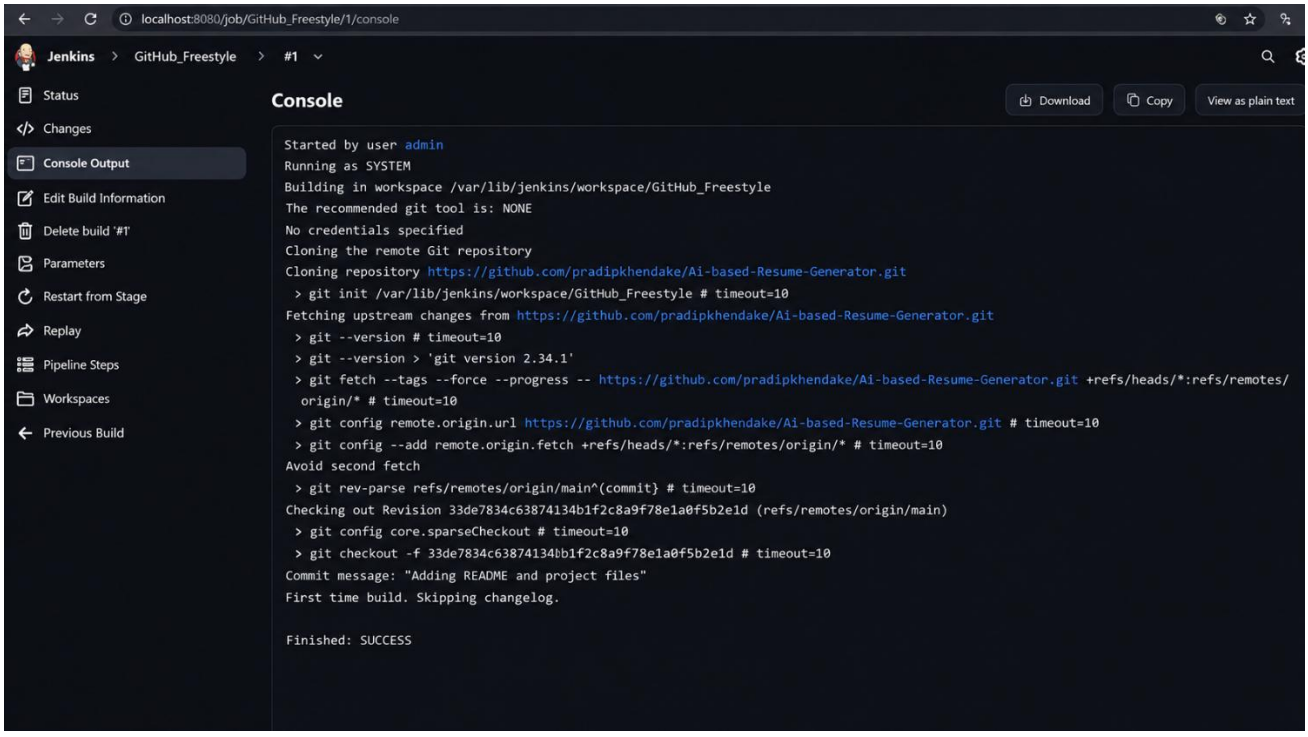



himanic72538 / a4_q3_web_app
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
a4_q3_web_app
Public
Pin
main
1 Branch
0 Tags
Go to file
T
Add file
Code
himanic72538 Add web app source code and Dockerfile
417b581 · 7 minutes ago
1 Commit
Dockerfile
Add web app source code and Dockerfile
7 minutes ago
app.py
Add web app source code and Dockerfile
7 minutes ago
README
Add a README
Help people interested in this repository understand your project.
Add a README

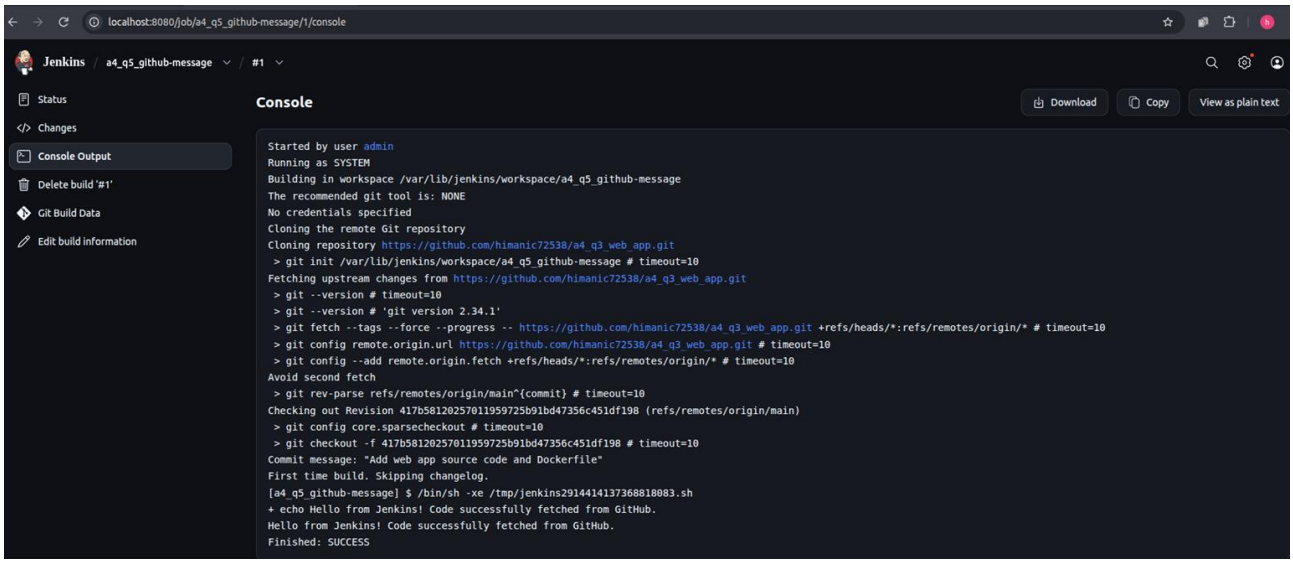

himanic72538 / a4_q3_web_app
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
Files
main
Go to file
T
Dockerfile
app.py
a4_q3_web_app / Dockerfile
himanic72538 Add web app source code and Dockerfile
Code
Blame
6 lines (6 loc) · 107 Bytes
1 FROM python:3.9-slim
2 WORKDIR /app
3 RUN pip install flask
4 COPY app.py .
5 EXPOSE 5000
6 CMD ["python", "app.py"]



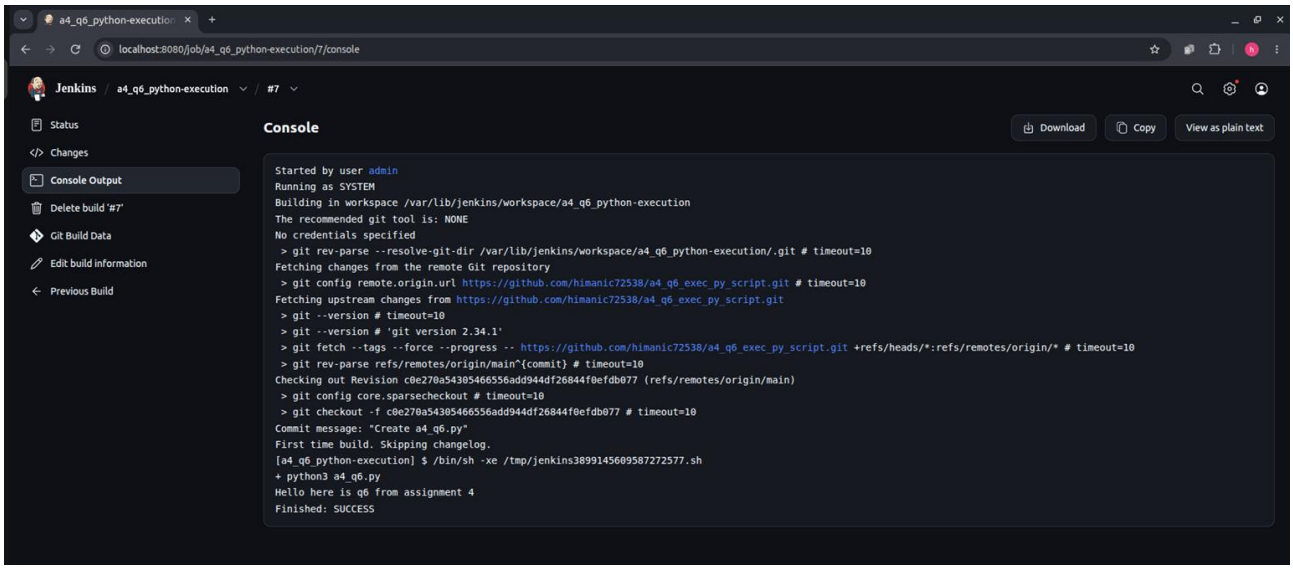
4\. Install and configure Jenkins. Create a Freestyle project that downloads source code from a GitHub repository.



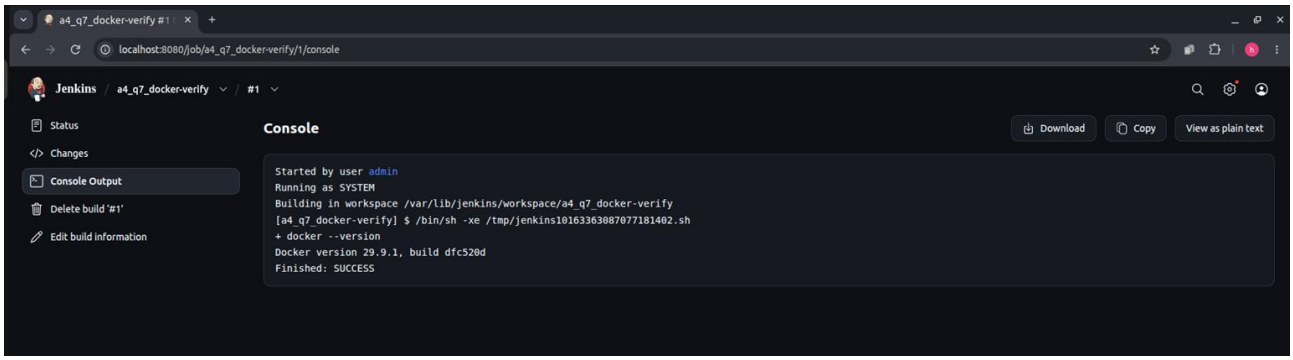
5\. Configure a Jenkins Freestyle project to fetch source code from GitHub and display a message in the Jenkins console output.



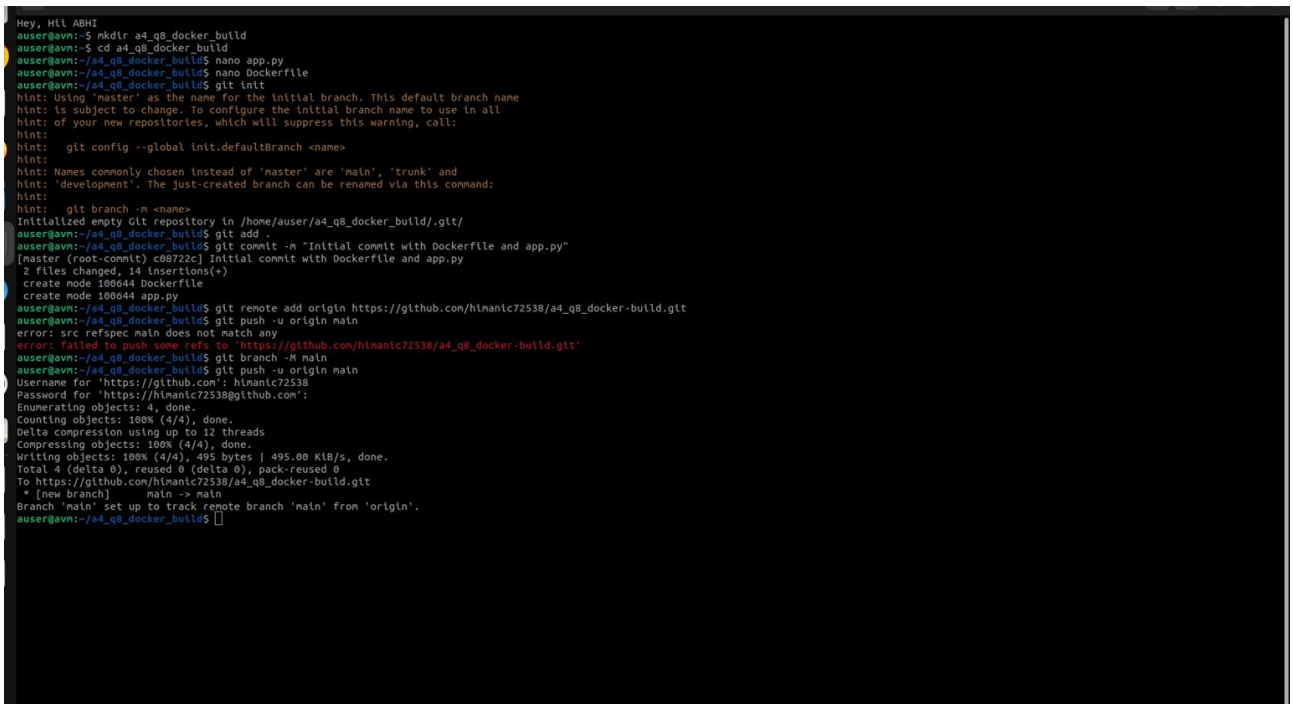
6\. Create a Jenkins job that checks out code from GitHub and executes a Python script.

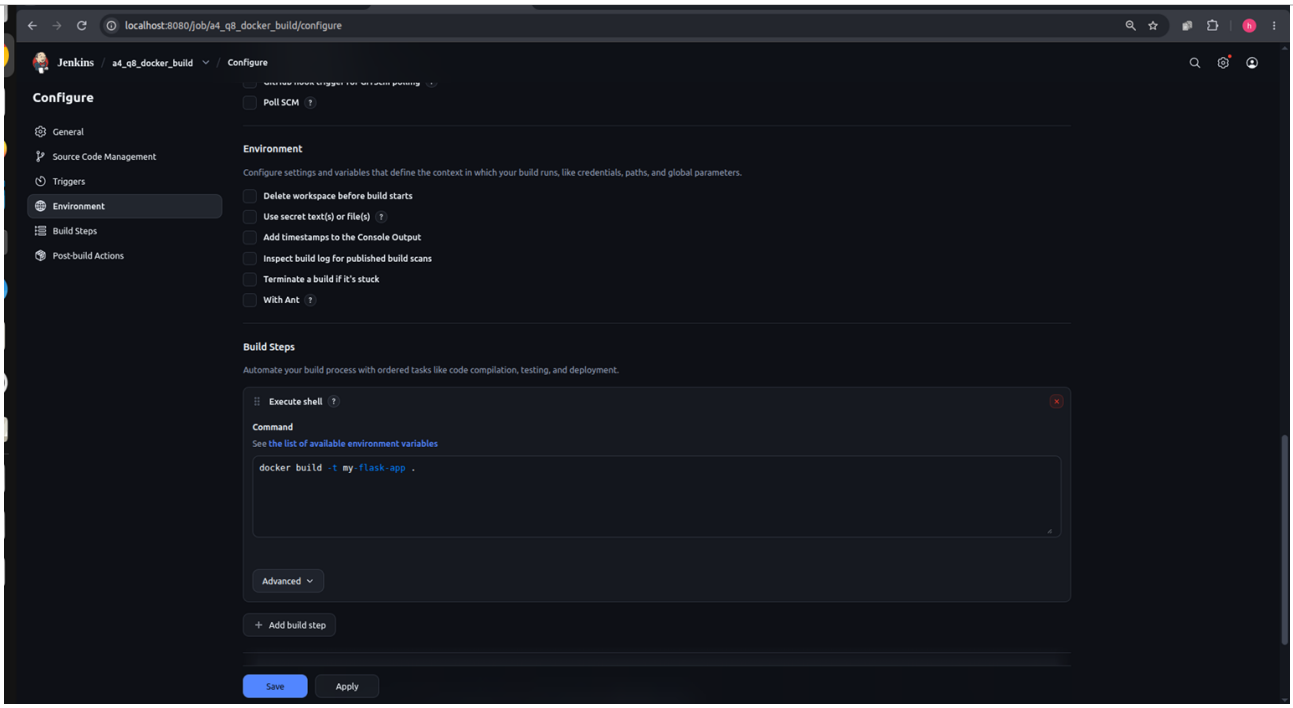


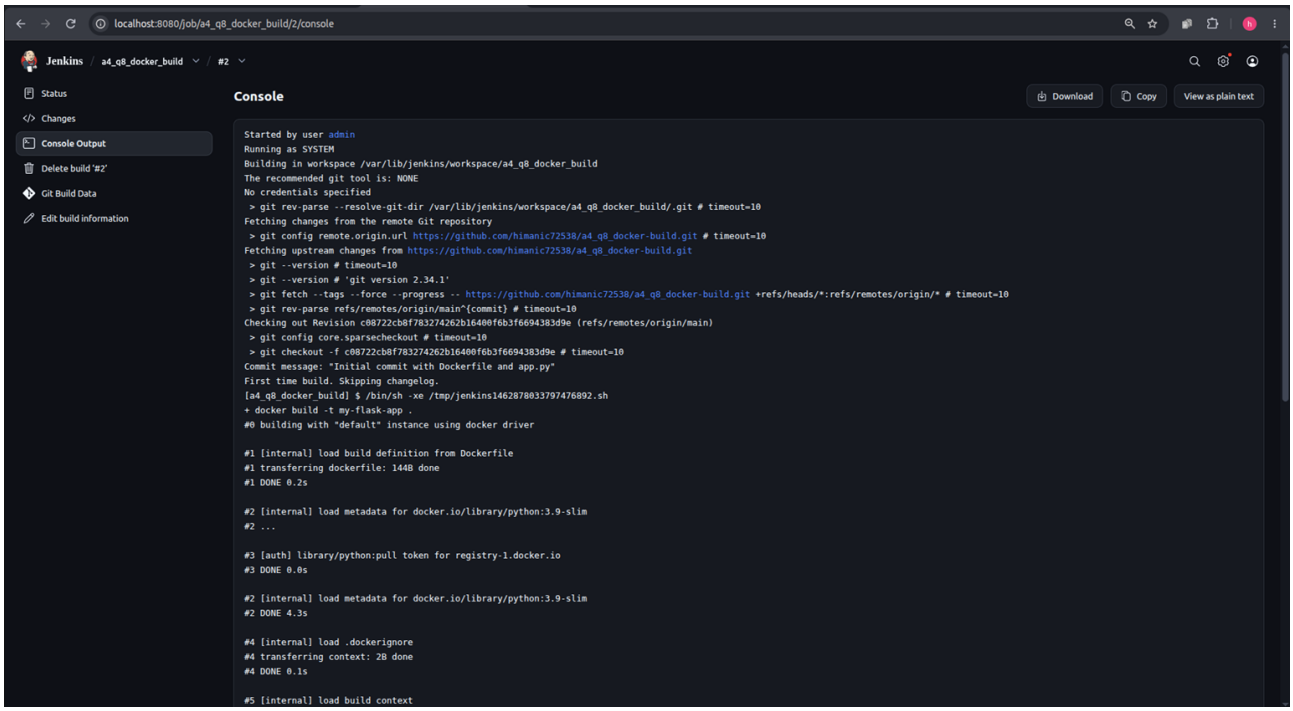
7\. Configure Jenkins to execute Docker commands. Verify the Docker installation by running `docker --version` through a Jenkins job.

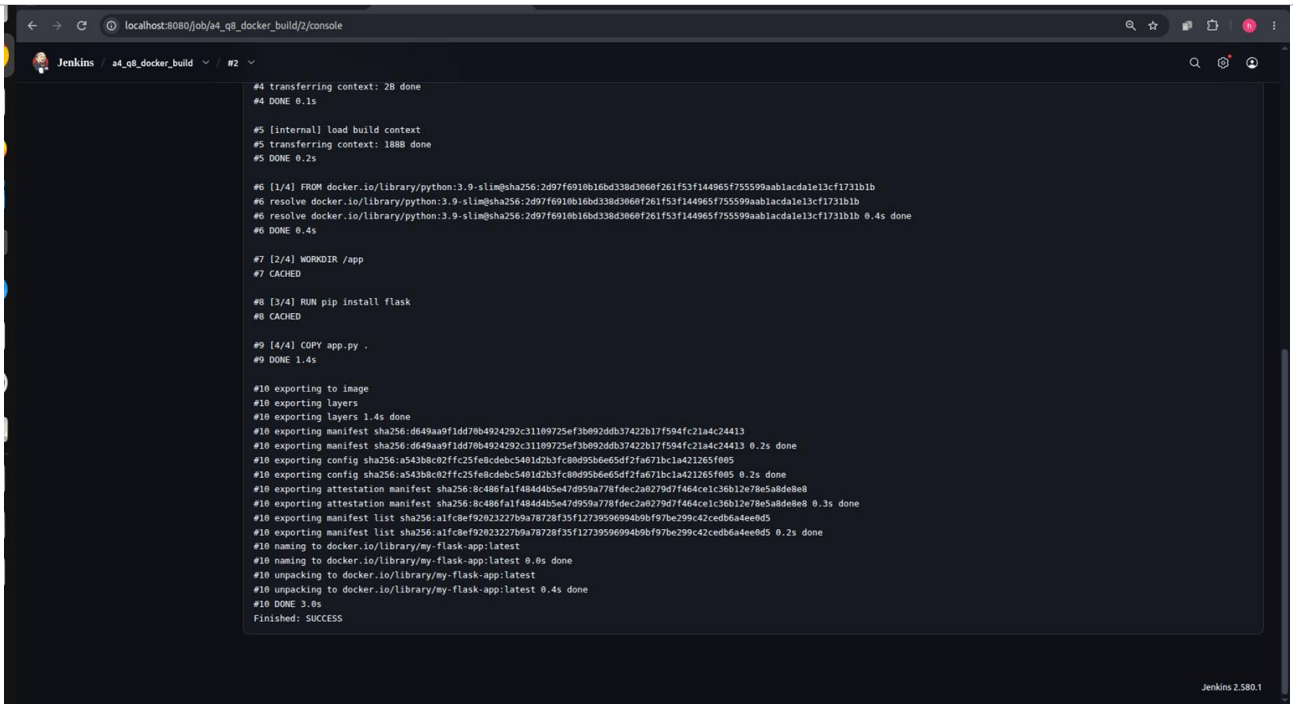


8\. Create a Jenkins job that fetches a GitHub repository and builds a Docker image using its `Dockerfile`.

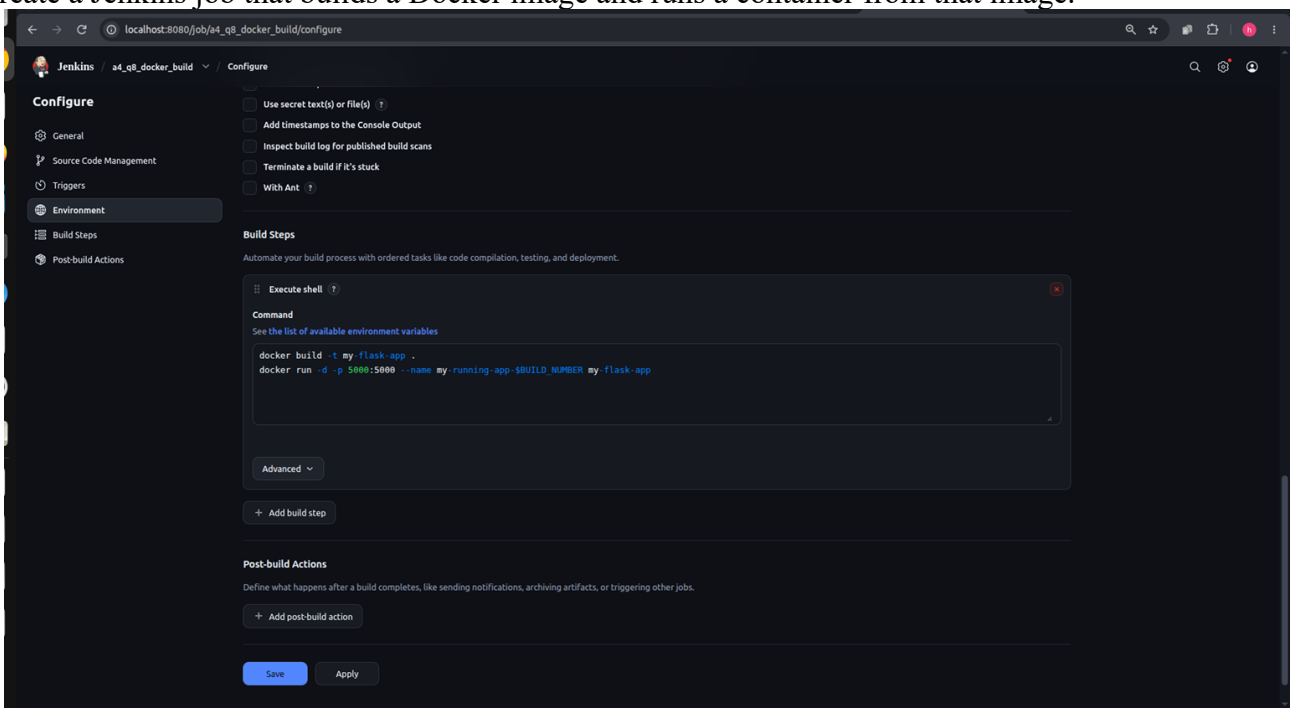


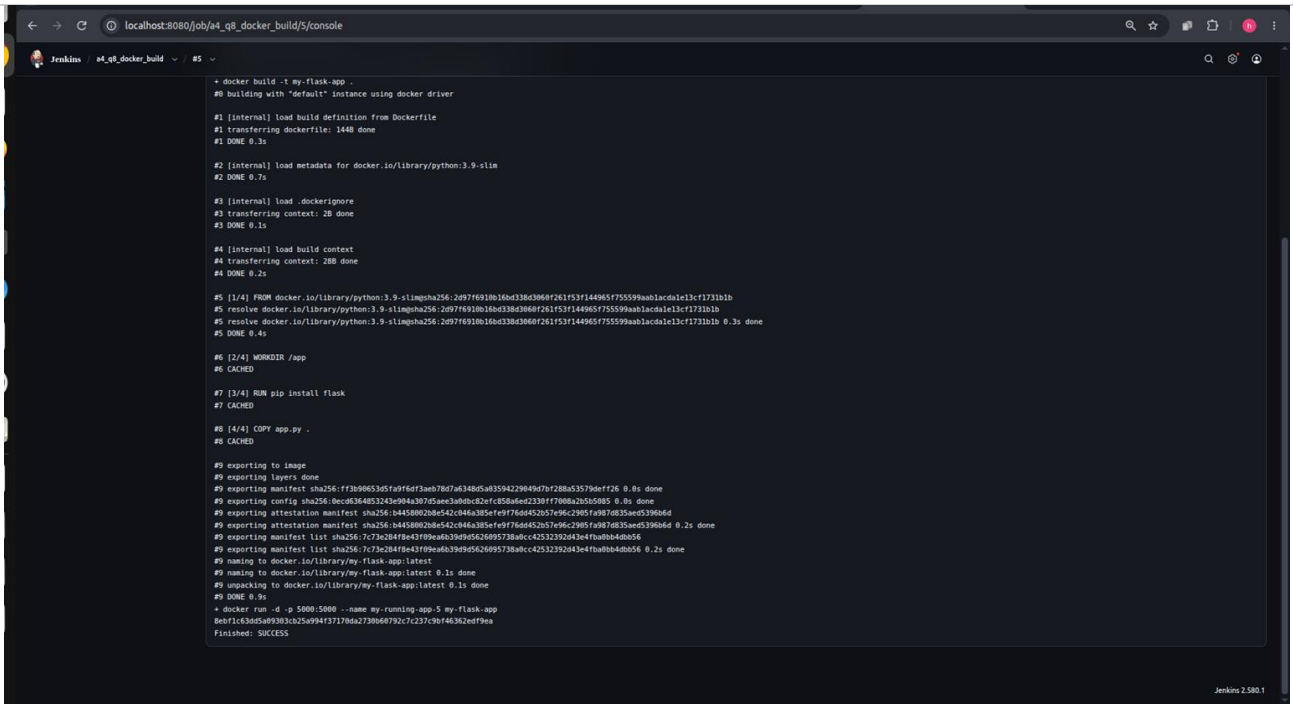

localhost:8080/job/a4_q8_docker_build/configure
Jenkins / a4_q8_docker_build / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Poll SCM
Environment
Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.
Delete workspace before build starts
Use secret text(s) or file(s)
Add timestamps to the Console Output
Inspect build log for published build scans
Terminate a build if it's stuck
With Ant
Build Steps
Automate your build process with ordered tasks like code compilation, testing, and deployment.
Execute shell
Command
See the list of available environment variables
docker build -t my-flask-app .
Advanced
Add build step
Save
Apply


localhost:8080/job/a4_q8_docker_build/2/console
Jenkins / a4_q8_docker_build / #2
Status
Changes
Console Output
Delete build '#2'
Git Build Data
Edit build information
Console
Download
Copy
View as plain text
Started by user admin
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/a4_q8_docker_build
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/a4_q8_docker_build/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/himanic72538/a4_q8_docker-build.git # timeout=10
Fetching upstream changes from https://github.com/himanic72538/a4_q8_docker-build.git
> git --version # timeout=10
> git --version # 'git version 2.34.1'
> git fetch --tags --force --progress -- https://github.com/himanic72538/a4_q8_docker-build.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision c08722cb8f783274262b16400f6b3f6694383d9e (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f c08722cb8f783274262b16400f6b3f6694383d9e # timeout=10
Commit message: "Initial commit with Dockerfile and app.py"
First time build. Skipping changelog.
[a4_q8_docker_build] $ /bin/sh -xe /tmp/jenkins14628803797476892.sh
+ docker build -t my-flask-app .
#0 building with "default" instance using docker driver
#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 144B done
#1 DONE 0.2s
#2 [internal] load metadata for docker.io/library/python:3.9-slim
#2 ...
#3 [auth] library/python:pull token for registry-1.docker.io
#3 DONE 0.0s
#2 [internal] load metadata for docker.io/library/python:3.9-slim
#2 DONE 4.3s
#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.1s
#5 [internal] load build context

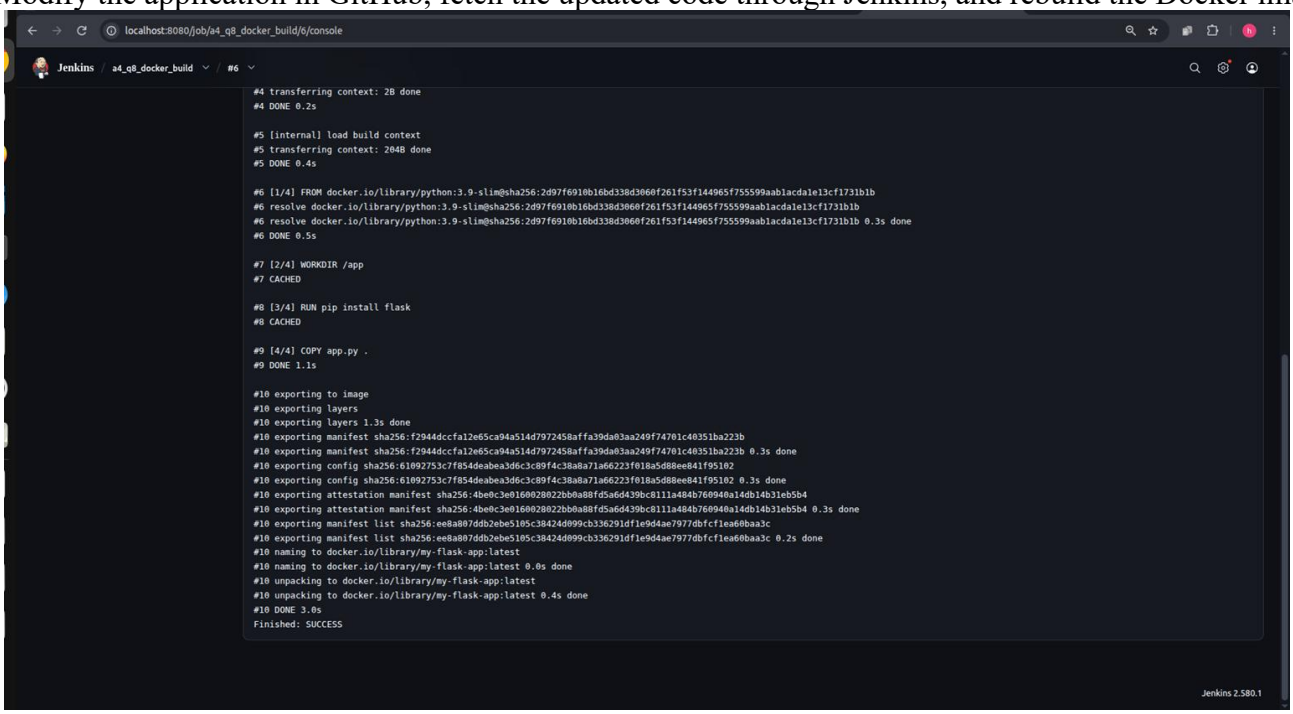


9\. Create a Jenkins job that builds a Docker image and runs a container from that image.

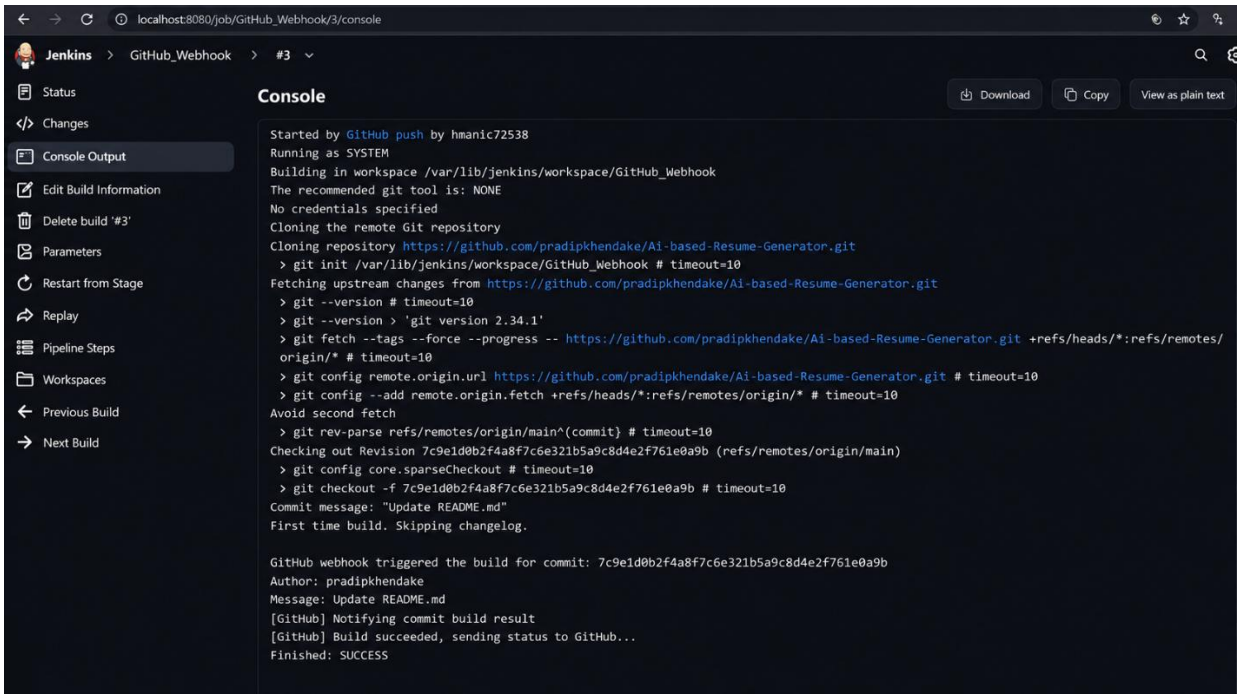




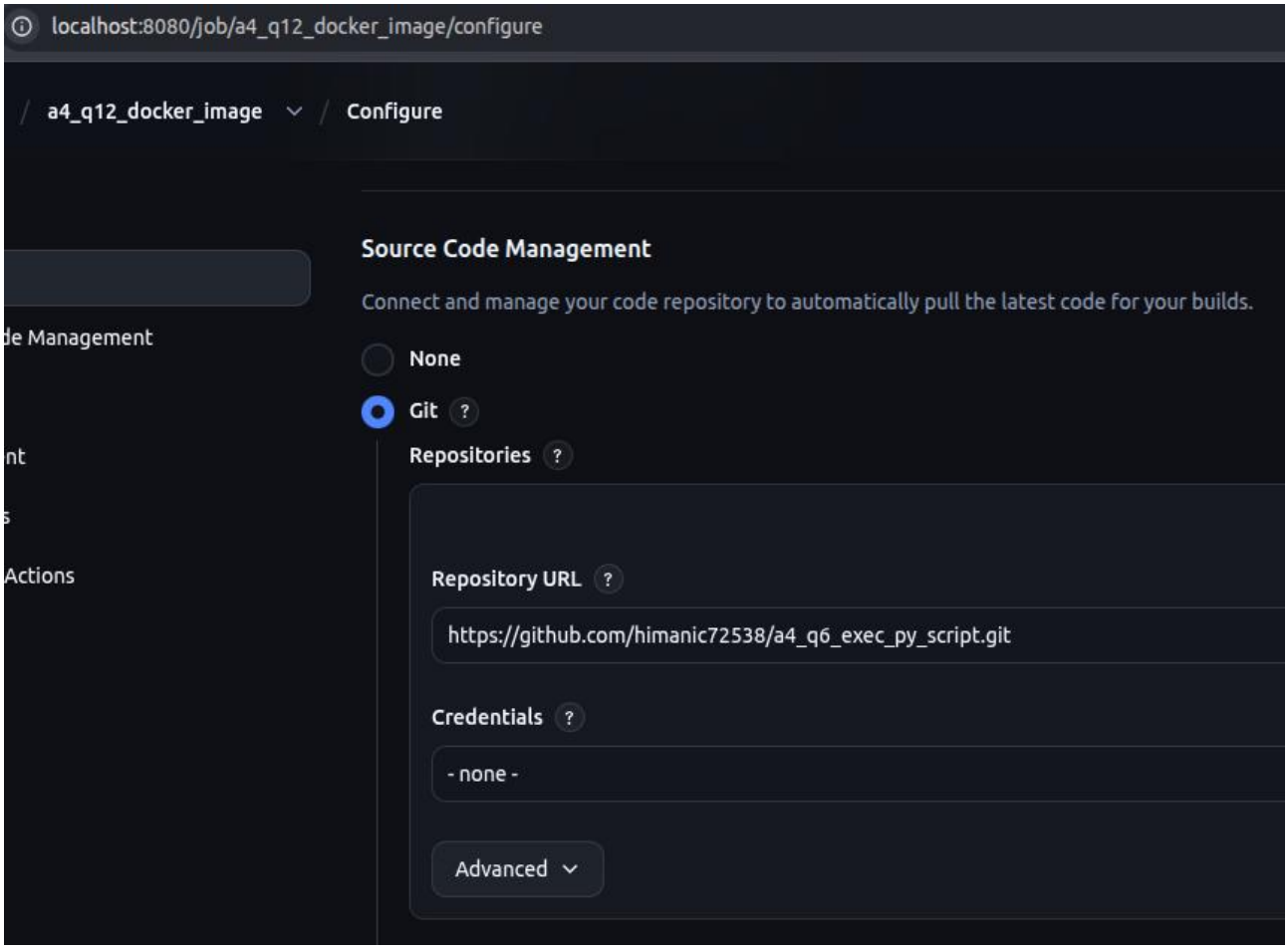
10\. Modify the application in GitHub, fetch the updated code through Jenkins, and rebuild the Docker image.

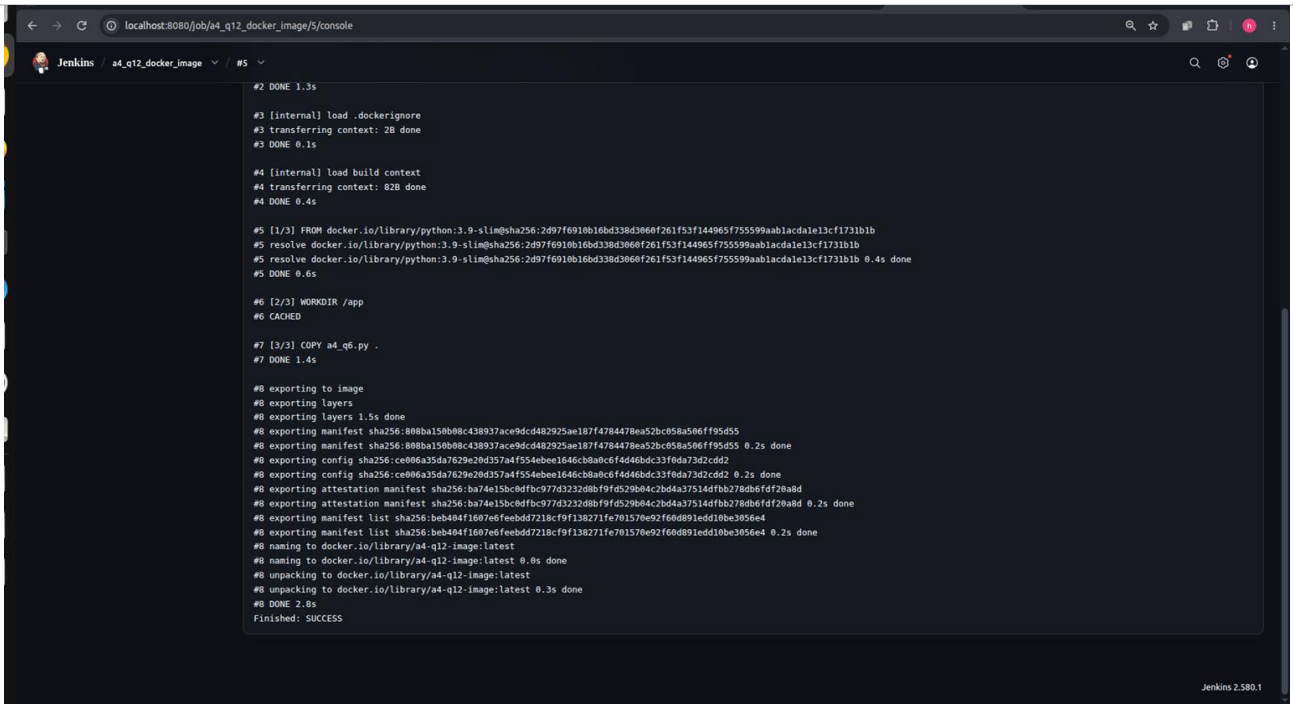


11\. Configure a GitHub webhook to trigger a Jenkins build whenever new code is pushed to the repository.



12\. Create a Jenkins Freestyle project that checks out code from a specified GitHub branch and builds a Docker image.





13\. Configure Jenkins to build a Docker image with a specified image name and tag, such as `myapp:v1`.

```
#5 DONE 0.3s

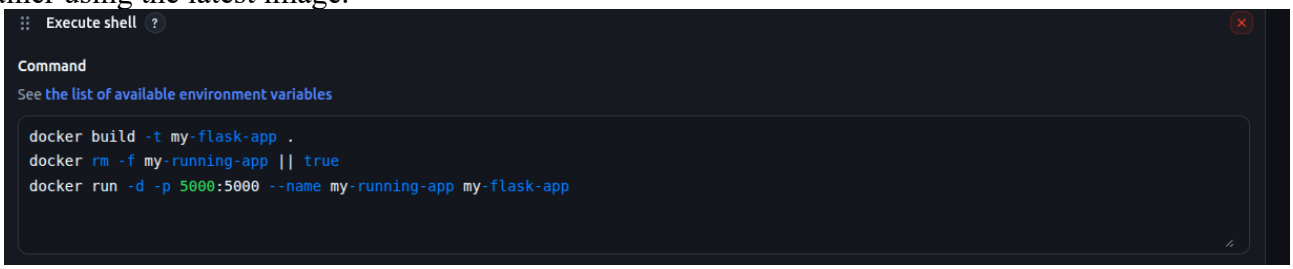
#6 [1/3] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aa
#6 resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1a
#6 resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1a
#6 DONE 0.5s

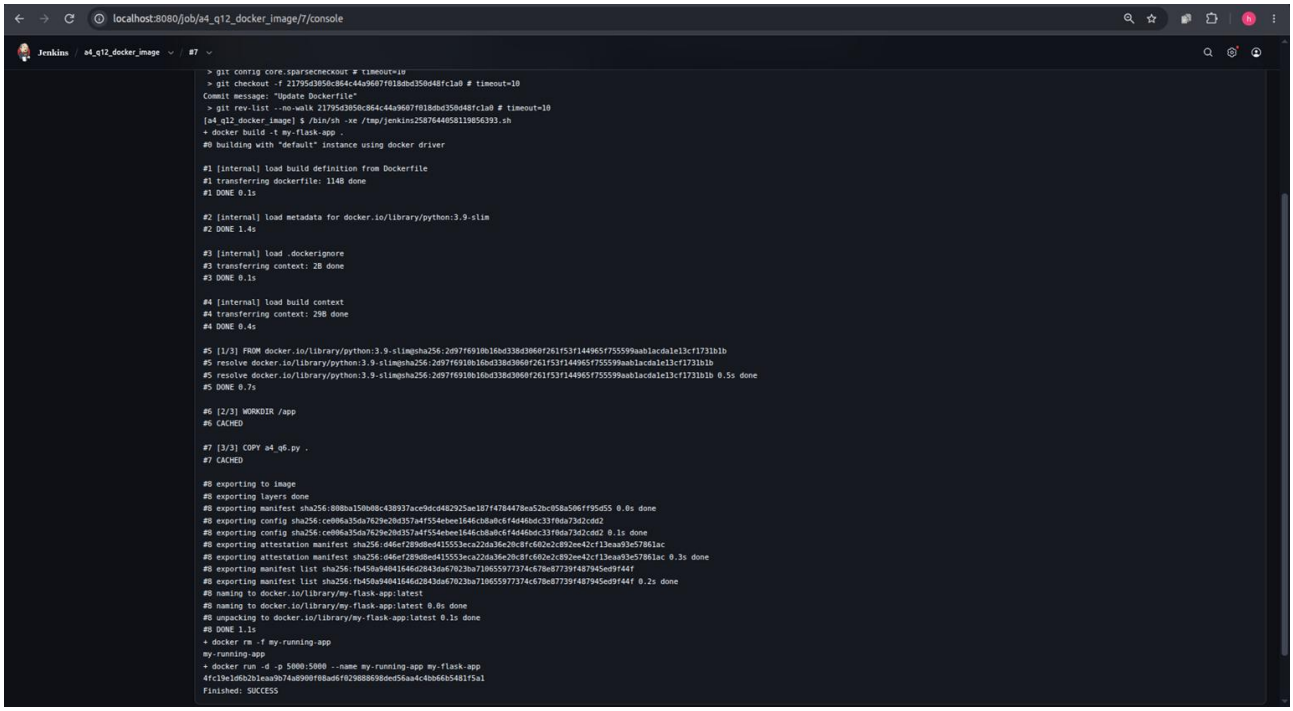
#7 [2/3] WORKDIR /app
#7 CACHED

#8 [3/3] COPY a4_q6.py .
#8 CACHED

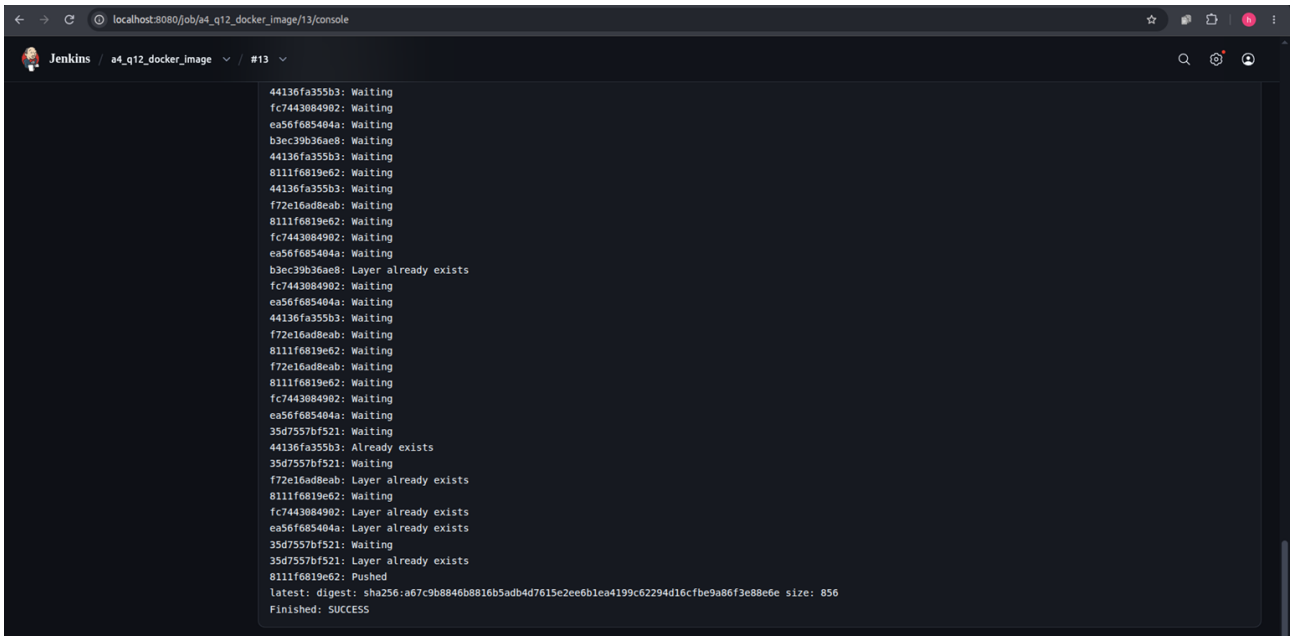
#9 exporting to image
#9 exporting layers done
#9 exporting manifest sha256:808ba150b08c438937ace9dcd482925ae187f4784478ea52bc058a506ff95d55 0.0s d
#9 exporting config sha256:ce006a35da7629e20d357a4f554ebee1646cb8a0c6f4d46bdc33f0da73d2cdd2
#9 exporting config sha256:ce006a35da7629e20d357a4f554ebee1646cb8a0c6f4d46bdc33f0da73d2cdd2 0.1s don
#9 exporting attestation manifest sha256:71b8238d5ee8c39dbf74cfc7ea25e3818ddd4c05e8220ae799fe8aba0a3
#9 exporting attestation manifest sha256:71b8238d5ee8c39dbf74cfc7ea25e3818ddd4c05e8220ae799fe8aba0a3
#9 exporting manifest list sha256:6b70fdc64b2f35f2abd22991e0315c5c52b5ba632f9141cbb17257ff69782d34
#9 exporting manifest list sha256:6b70fdc64b2f35f2abd22991e0315c5c52b5ba632f9141cbb17257ff69782d34 0
#9 naming to docker.io/library/myapp:v1
#9 naming to docker.io/library/myapp:v1 0.0s done
#9 unpacking to docker.io/library/myapp:v1 0.1s done
#9 DONE 1.0s
Finished: SUCCESS
```

14\. Create a Jenkins job that stops and removes an existing application container before starting a new container using the latest image.

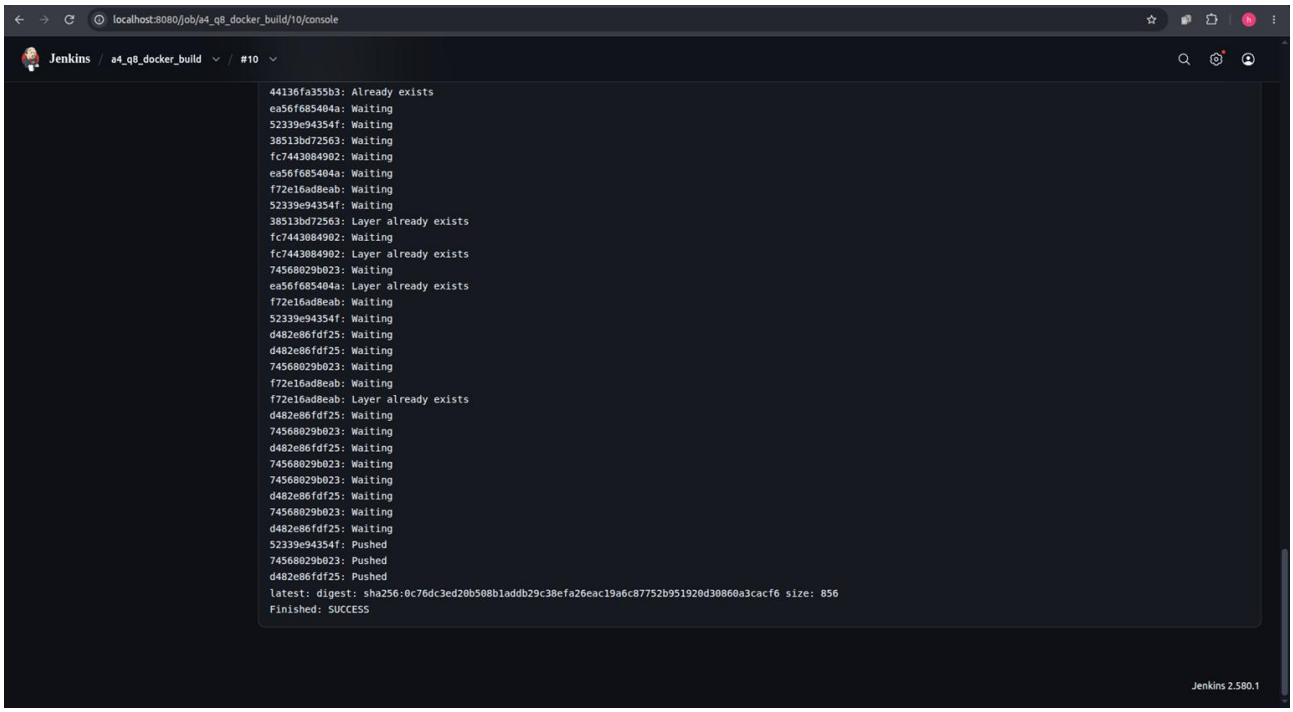




15\. Configure Jenkins to log in to Docker Hub and push a Docker image built from a GitHub repository.

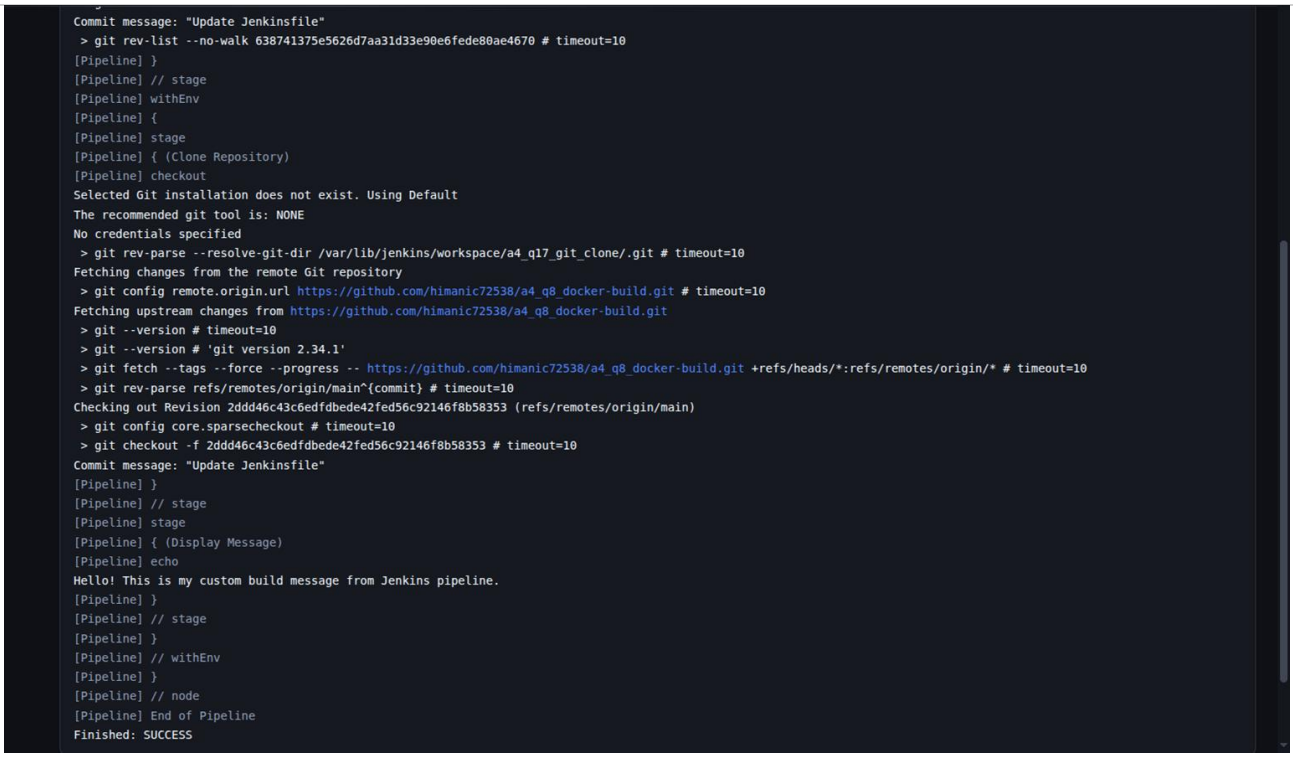


16\. Create a Jenkins job that checks out application code, builds a Docker image, and pushes the image to Docker Hub.

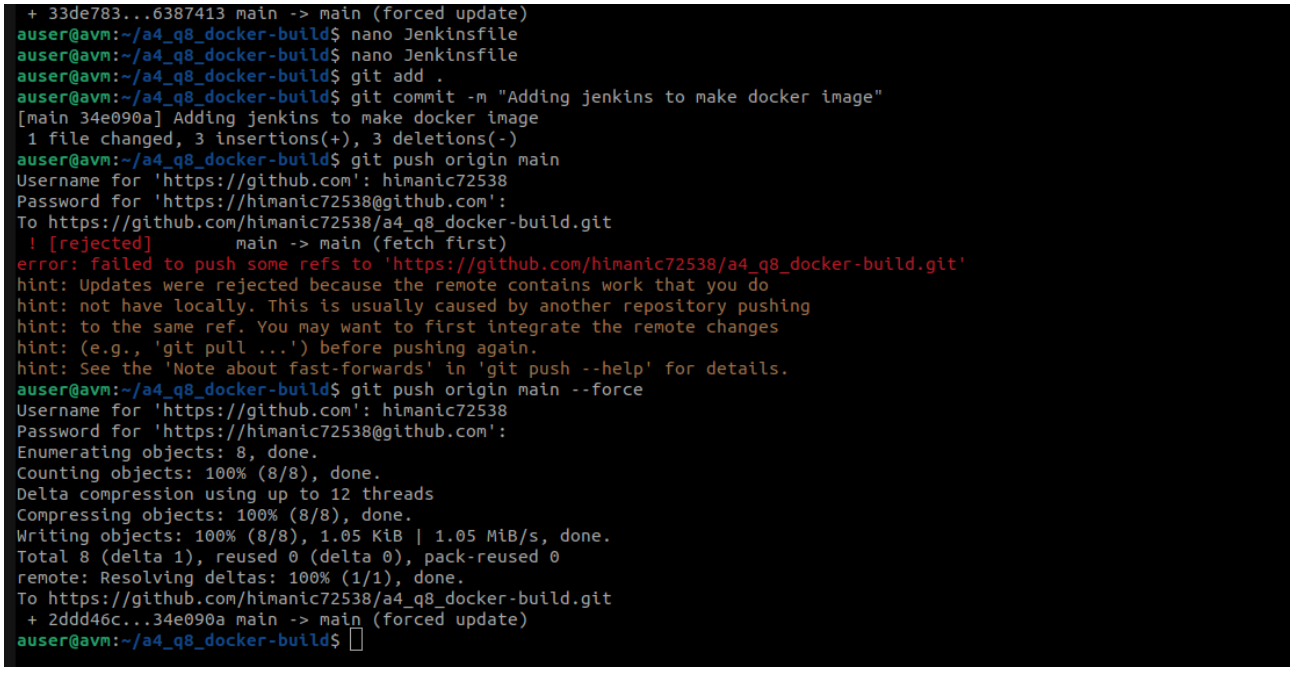


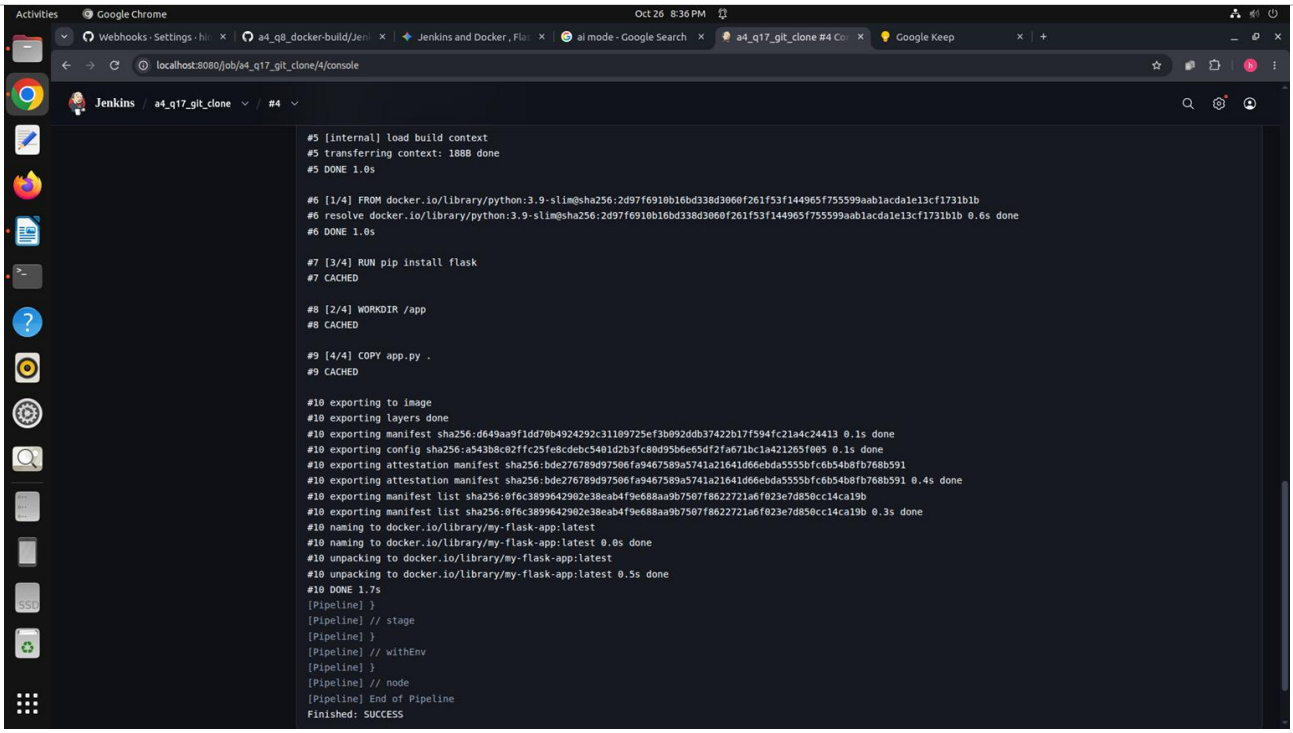
17\. Write a simple `Jenkinsfile` containing stages to clone a GitHub repository and display a build message.



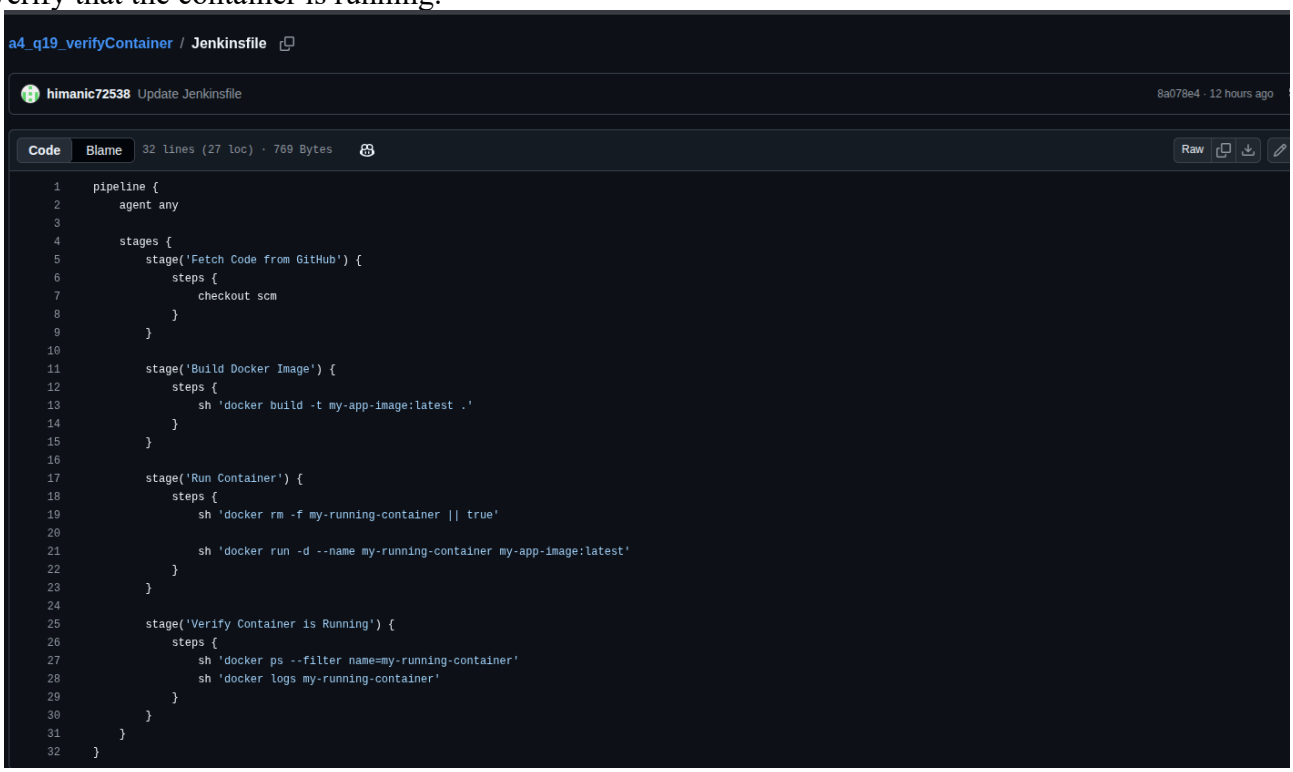


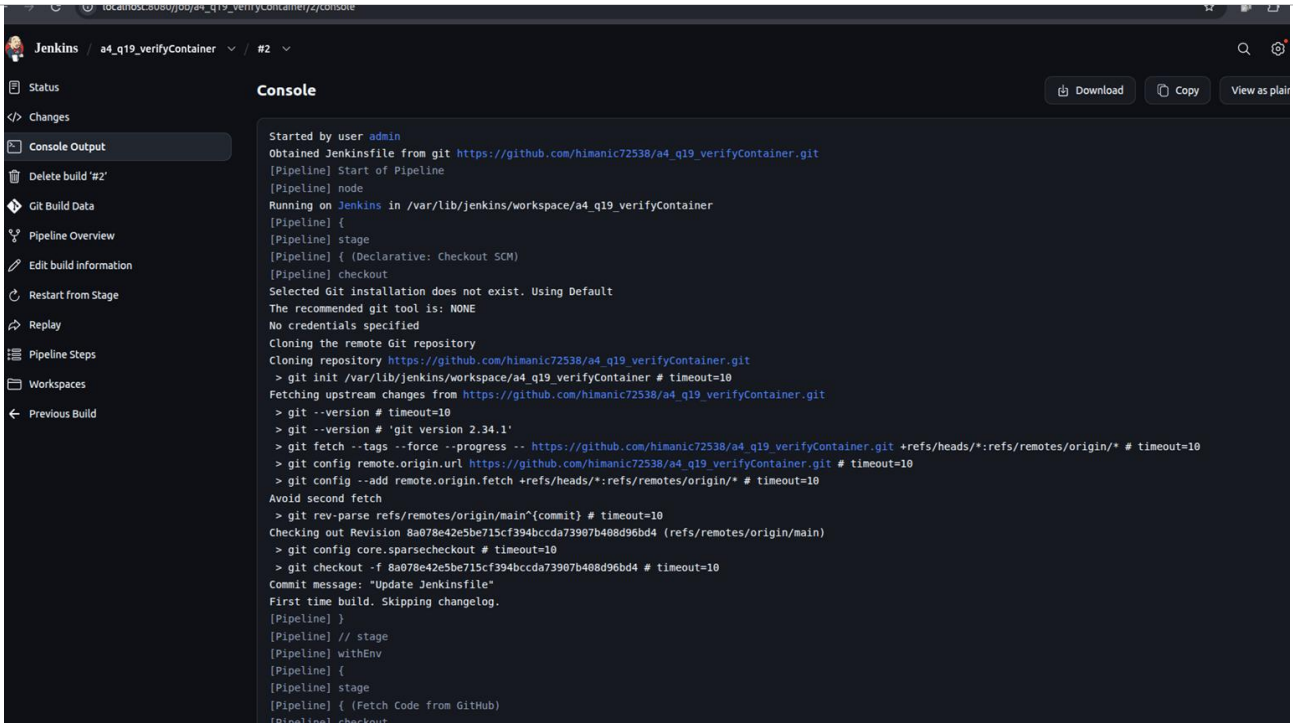
18\. Extend the `Jenkinsfile` to include stages for checking out code from GitHub and building a Docker image.

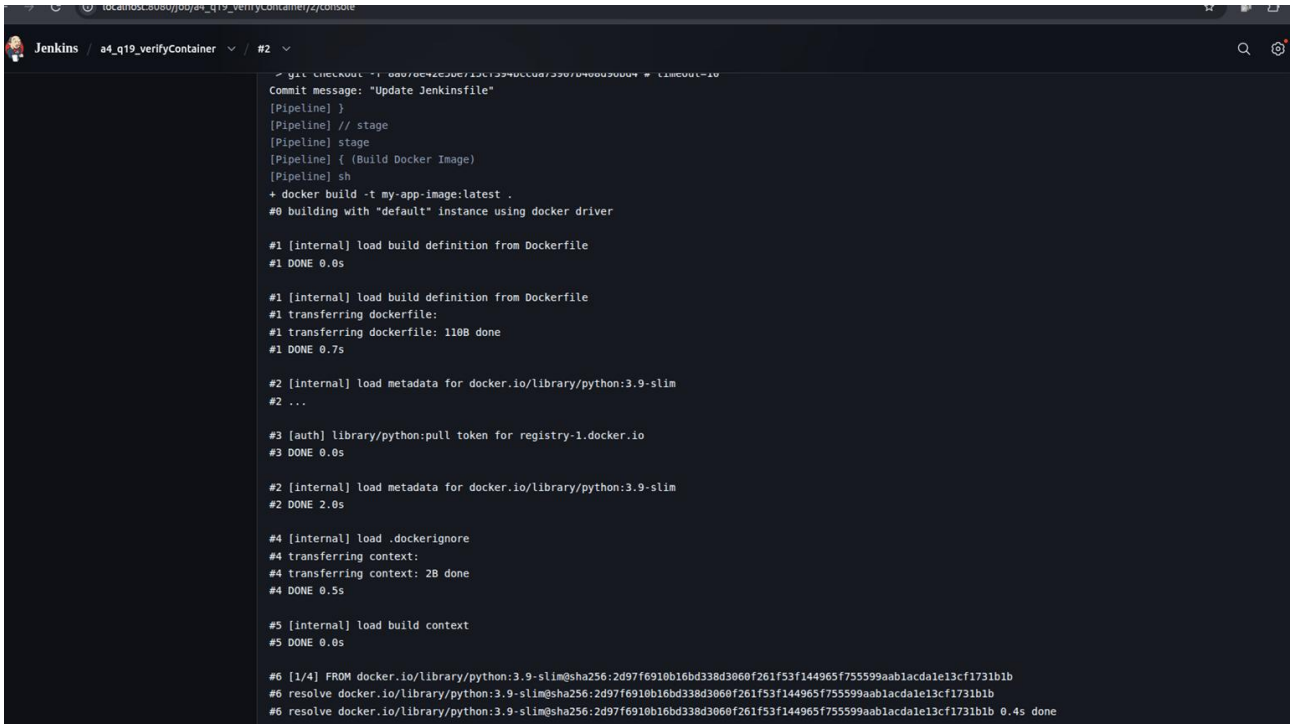


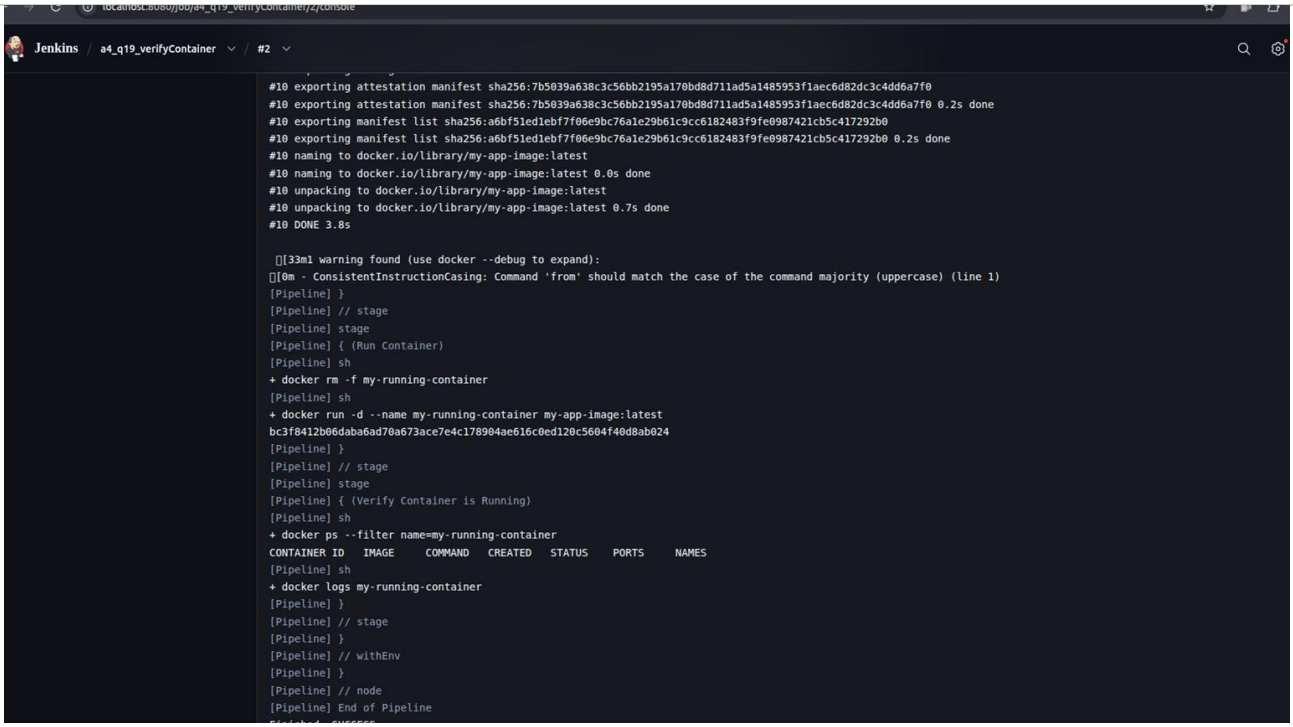


19\. Create a Jenkins pipeline with stages to fetch code from GitHub, build a Docker image, run a container, and verify that the container is running.

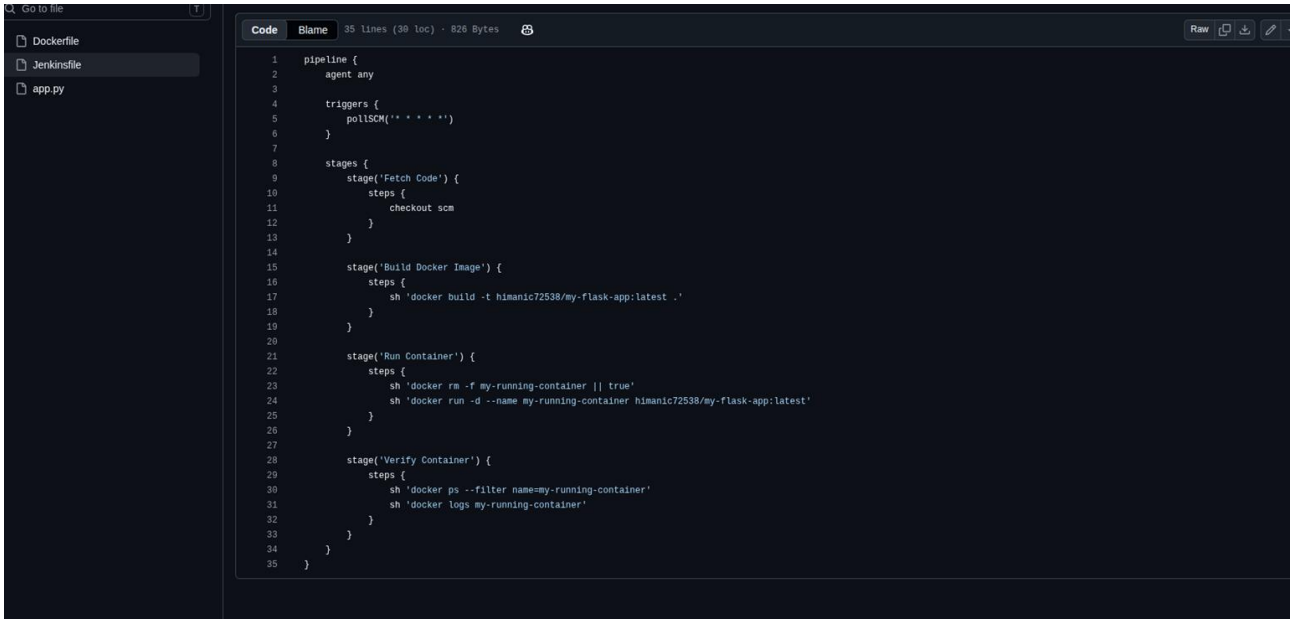


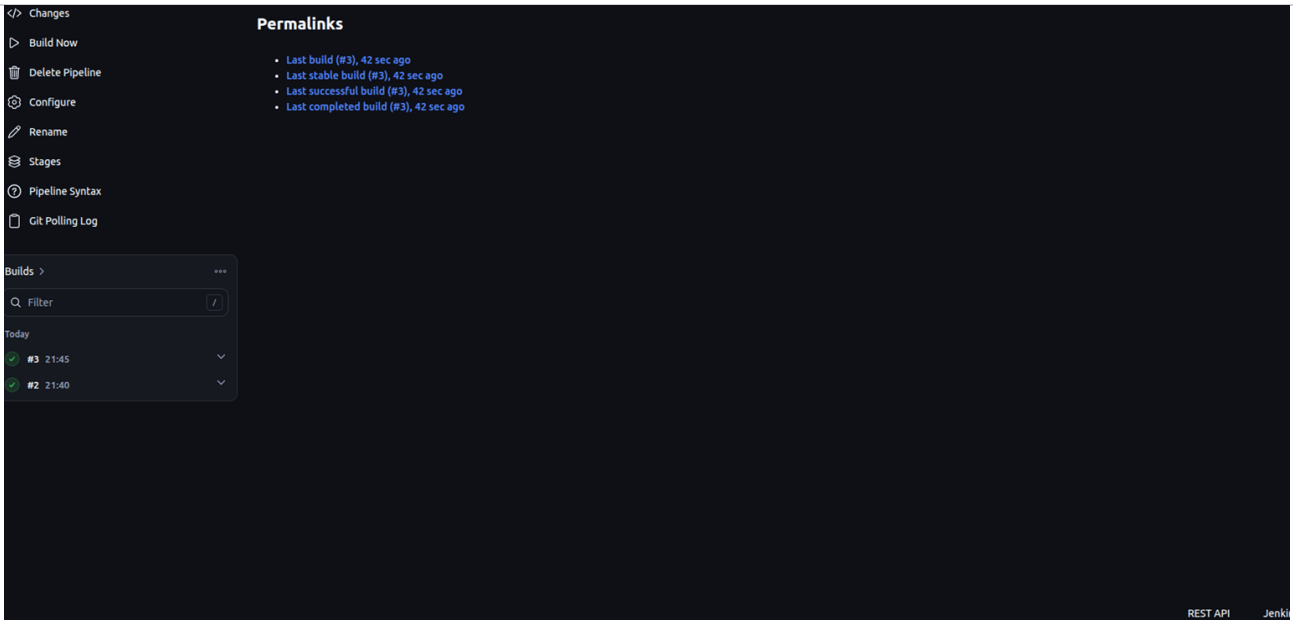

Jenkins
a4_q19_verifyContainer
#2
Status
Changes
Console Output
Delete build '#2'
Git Build Data
Pipeline Overview
Edit build information
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build
Console
Download
Copy
Started by user admin
Obtained Jenkinsfile from git https://github.com/himanic72538/a4_q19_verifyContainer.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/a4_q19_verifyContainer
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/himanic72538/a4_q19_verifyContainer.git
> git init /var/lib/jenkins/workspace/a4_q19_verifyContainer # timeout=10
Fetching upstream changes from https://github.com/himanic72538/a4_q19_verifyContainer.git
> git --version # timeout=10
> git --version # 'git version 2.34.1'
> git fetch --tags --force --progress -- https://github.com/himanic72538/a4_q19_verifyContainer.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/himanic72538/a4_q19_verifyContainer.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 8a078e42e5be715cf394bccda73907b408d96bd4 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 8a078e42e5be715cf394bccda73907b408d96bd4 # timeout=10
Commit message: "Update Jenkinsfile"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Fetch Code from GitHub)


Jenkins
a4_q19_verifyContainer
#2
Commit message: "Update Jenkinsfile"
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build Docker Image)
[Pipeline] sh
+ docker build -t my-app-image:latest .
#0 building with "default" instance using docker driver
#1 [internal] load build definition from Dockerfile
#1 DONE 0.0s
#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile:
#1 transferring dockerfile: 110B done
#1 DONE 0.7s
#2 [internal] load metadata for docker.io/library/python:3.9-slim
#2 ...
#3 [auth] library/python:pull token for registry-1.docker.io
#3 DONE 0.0s
#2 [internal] load metadata for docker.io/library/python:3.9-slim
#2 DONE 2.0s
#4 [internal] load .dockerignore
#4 transferring context:
#4 transferring context: 2B done
#4 DONE 0.5s
#5 [internal] load build context
#5 DONE 0.0s
#6 [1/4] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b
#6 resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b
#6 resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b 0.4s done



20\. Implement a simple CI/CD pipeline using GitHub, Jenkins, and Docker. Whenever application code is pushed to GitHub, Jenkins should fetch the code, build a new Docker image, and run the updated application in a container..




Changes
Build Now
Delete Pipeline
Configure
Rename
Stages
Pipeline Syntax
Git Polling Log
Builds
Filter
Today
#3 21:45
#2 21:40
Permalinks
Last build (#3), 42 sec ago
Last stable build (#3), 42 sec ago
Last successful build (#3), 42 sec ago
Last completed build (#3), 42 sec ago
REST API